

Allyson Frailey
April 21st 2026 Lab
Tuesdays @ 5:15pm

Lab 3 - Two Level Circuit Design

Lab Report

1. Introduction

Lab Partner: Priyanka Burra

The goal of this project was to make a circuit that controls traffic lights for three lanes: a carpool lane and two regular lanes (left and right) .

How the circuit works:

- Sensors: We used sensors (CS, LS, RS) to see if cars were waiting and a signal (RR) to decide which lane goes next .
- Priority: If a car is in the carpool lane, it always gets a green light first.
- Alternating: If the carpool lane is empty, the green light swaps between the left and right lanes using a "round robin" style.
- Default: If no cars are there at all, the right light stays green.

Design Choice: We chose the SOP design because it was easier for us to build, it was the more "expensive" route so we should've used POS .

2. Schematic and Testing

Schematic: We drew the circuit using AND and OR gates to match our SOP equations.

SOP Equations Used:

Minimized SOP:

- $CL = CS$
- $LL = CS'LSRS' + CS'LSRR'$
- $RL = CS'LS' + CS'RSRR$

7-Segment Display: We added a module to show how many sensors (CS, LS, RS) were active, displaying a number from 0 to 3.

FPGA: We programmed the design onto an Intel FPGA, using switches for inputs and LEDs for the traffic lights .

3. Simulation Strategy

We tested the design by simulating all 16 possible input combinations (2^4). This allowed us to check every scenario from our truth table on the computer before using the hardware. This strategy helps find "logic errors," like putting an AND gate where an OR gate should be or accidentally flipping a signal.

4. Why Simulation Works

Because this circuit only has four inputs, the simulation is very effective. We can test 100% of the possible states. If the simulation output matches our truth table, we can be confident the logic is correct before downloading it to the FPGA.

5. FPGA vs. Computer Tests

Even if the simulation is perfect, we might find errors on the FPGA board. These are usually "hardware errors" or "setup errors," like:

- Assigning the wrong pin numbers to the switches or LEDs.
- Loose wires or a broken switch on the board.
- The simulation shows the math is right, but the physical board tests if the parts are connected correctly.

6. Error Light (ERR1)

To add a new light (ERR1) that turns on if two or more lights are green at once, the logic would be: $ERR1 = (CL \cdot LL) + (CL \cdot RL) + (LL \cdot RL)$

Similarity to T1:

This is similar to the T1 logic used for the car counter. T1 is the "tens" bit of our count and turns on when two or three sensors are active. Both use "at least two" logic to determine the output.

7. Adding a Timer (TM)

To add a timer (\$TM\$) that turns all lights red when it is "0," we would use AND gates as "enables".

- When $TM=1$, the lights work normally.
- When $TM=0$, the output of the AND gate will always be 0 (red).

In the schematic, each light output (CL, LL, RL) would go into a 2-input AND gate. The other input for all three gates would be the TM signal.

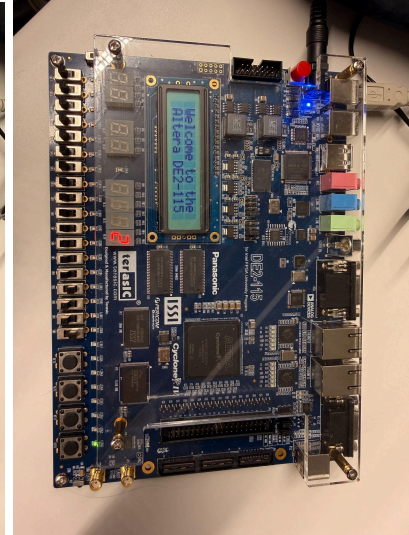
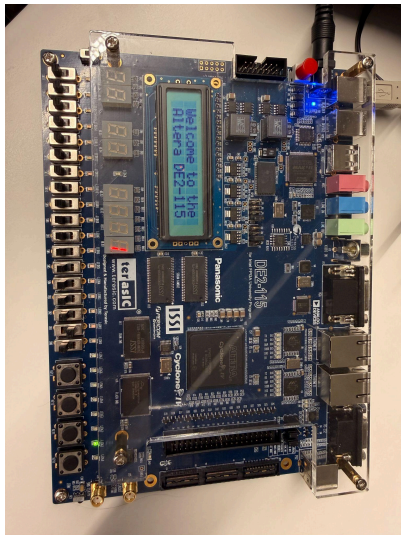
8. Better Traffic Strategy

Right now, the carpool lane has total priority, which could trap regular cars in a long line forever. A better strategy would be:

- **Time Sharing:** Give the carpool lane priority for a maximum amount of time (like 1 minute), then force a change to the other lanes.

- **Sensor Weighting:** If the non-carpool lanes have many cars waiting while the carpool lane only has one, the system could rotate the lights more fairly to keep traffic moving for everyone.

Lab Photos

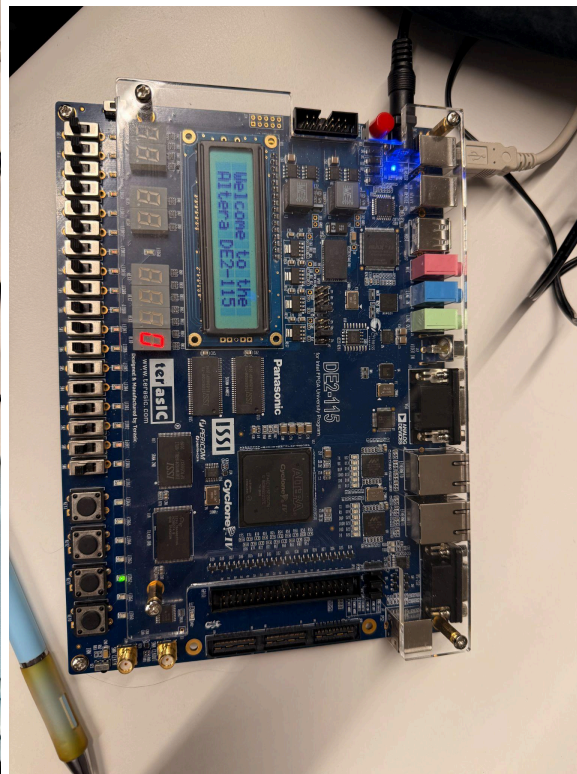
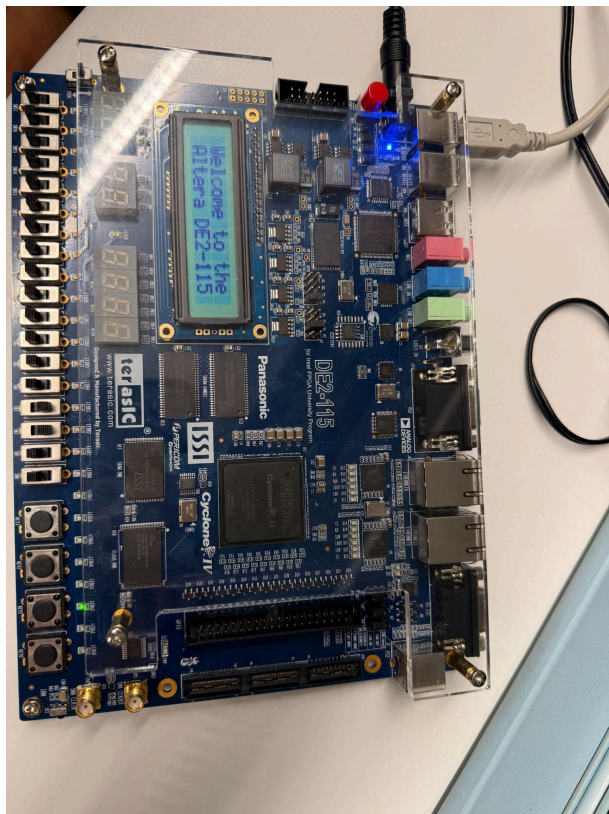
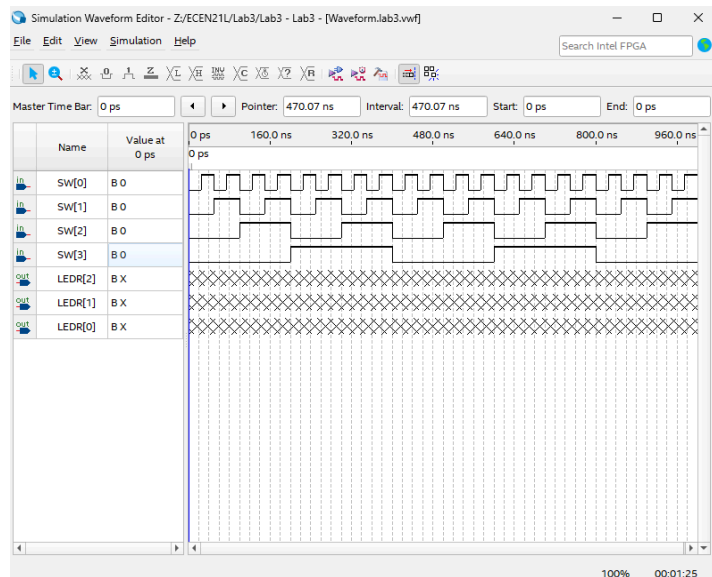


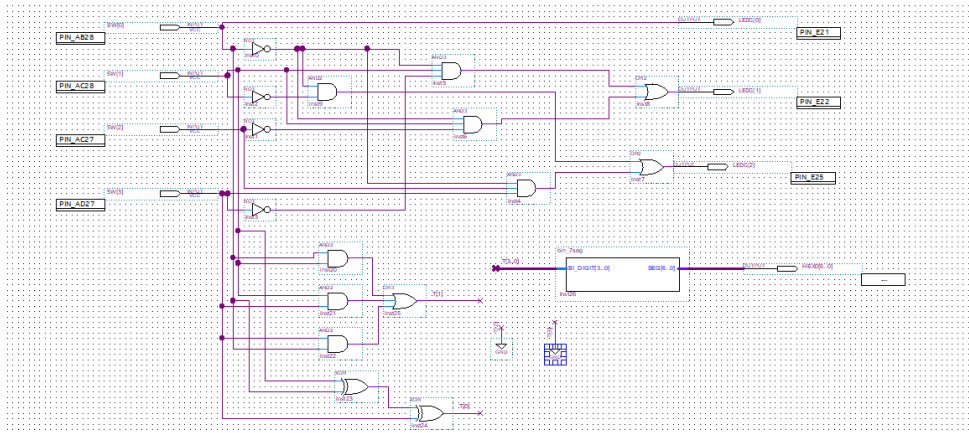
```

always @(BI_DIGIT)
  case (BI_DIGIT)
    4'h0: SEG = ~7'b0111111;
    4'h1: SEG = ~7'b0000110;
    4'h2: SEG = ~7'b1011011;
    4'h3: SEG = ~7'b1001111;
    4'h4: SEG = ~7'b1100110;
    4'h5: SEG = ~7'b1101101;
    4'h6: SEG = ~7'b1111100;
    4'h7: SEG = ~7'b0000111;
    4'h8: SEG = ~7'b1111111;
    4'h9: SEG = ~7'b1100111;
    4'ha: SEG = ~7'b1110111;
    4'hb: SEG = ~7'b1111100;
    4'hc: SEG = ~7'b1011000;
    4'hd: SEG = ~7'b1011110;
    4'he: SEG = ~7'b1111001;
    4'hf: SEG = ~7'b1110001;
  endcase
endmodule

```

CS	RS	LS	T ₁	T ₀
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1





Simulation Flow Progress

Error

Info: the Altera IP License Agreement, or other applicable license
 Info: agreement, including, without limitation, that your use is for
 Info: the sole purpose of programming logic devices manufactured by
 Info: Altera and sold by Altera or its authorized distributors. Please
 Info: refer to the Altera Software License Subscription Agreements
 Info: on the Quartus Prime software download page.
 Info: Processing started: Tue Apr 21 18:07:42 2026

Info: Command: quartus_eda --gen_testbench --tool=modelsim_oem --format=verilog --
 write_settings_files=off Lab3 -c Lab3 --vector_source=Z:/ECEN21L/Lab3/Waveform.vwf --
 testbench_file=Z:/ECEN21L/Lab3/simulation/qsim/Waveform.vwf.vt

Warning (18236): Number of processors has not been specified which may cause overloading on shared
 machines. Set the global assignment NUM_PARALLEL_PROCESSORS in your QSF to an appropriate value
 for best performance.

Error (199014): Vector source file Z:/ECEN21L/Lab3/Waveform.vwf specified with --
 testbench_vector_input_file option does not exist

Error: Quartus Prime EDA Netlist Writer was unsuccessful. 1 error, 1 warning

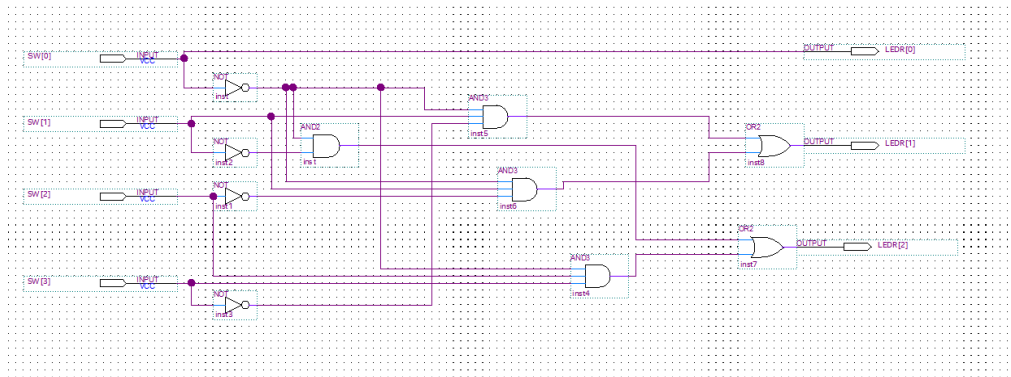
Error: Peak virtual memory: 4681 megabytes

Error: Processing ended: Tue Apr 21 18:07:43 2026

Error: Elapsed time: 00:00:01

Error: Total CPU time (on all processors): 00:00:01

Error.



A \ B	0	1
0	0	1
1	2	3

(a)

A \ BC	00	01	11	10
0	0	1	3	2
1	4	5	7	6

(b)

AB \ CD	00	01	11	10
00	0	1	3	2
01	4	5	7	6
11	12	13	15	14
10	8	9	11	10

(c)

Figure 1 Karnaugh Maps for (a) Two Variables (b) Three Variables (c) Four Variable:

Changed it from LEDR to LEDG